



School of Future Tech

Case Study Report

on

QuickBite: Food Delivery Order & Restaurant Management System

DBMS Case Study #83 · B.Tech Computer Science Engineering

by

Aditya Sunil Chouksey

Roll Number: 150096725070

Semester III · Academic Year 2025–2029

Index

1. Introduction to the Case Study.	3
2. Problem Statement / Case Background (Abstract).	4
3. Problem Statement / Case Study Design.	5
4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study.	7
5. Problem Statement / Case Study Implementation Details and Snapshots.	8
6. Problem Statement / Case Study Results and Conclusion.	11
7. References	12

1. Introduction to the Case Study

Online food delivery aggregators and restaurant platforms are central to urban commerce and daily consumer life, handling millions of transactional events across restaurants, dynamic menus, customer requests, and delivery logistics. Modern food delivery platforms such as Zomato, Swiggy, and DoorDash operate as high-throughput, multi-sided digital marketplaces. They continuously orchestrate real-time interactions across four key stakeholders: end consumers searching menus and submitting digital orders, partner restaurants managing dish availability and processing culinary preparation, freelance delivery riders fulfilling dispatch across dynamic geographical transit zones, and platform administrators tracking financial settlements, commissions, and marketing campaigns.

Underlying this high-frequency user ecosystem is a demanding database engineering requirement. A production food-delivery database cannot merely act as a passive data store; it must guarantee absolute transactional and financial consistency, enforce rigorous domain integrity across distributed entities, execute complex business intelligence aggregations, and sustain sub-millisecond query search response times under catalogs spanning hundreds of thousands of restaurants.

This case study focuses on designing, architecting, and implementing **QuickBite** — a relational database management system for a food delivery order and restaurant aggregator platform. Engineered and benchmarked using **PostgreSQL 18**, QuickBite demonstrates how foundational database theory — spanning conceptual Entity-Relationship (ER) modeling, mathematical dependency decomposition up to Boyce-Codd Normal Form (BCNF), procedural PL/pgSQL triggers, non-materialized views, composite B-Tree indexing, and advanced SQL analytical queries — solves the operational bottlenecks inherent in large-scale online food commerce.

2. Problem Statement / Case Background (Abstract)

Background

In early-stage platform development or poorly architected architectures, delivery applications frequently record restaurant menus, customer accounts, and order transactions in flat, unnormalized tables or disjointed spreadsheets. In an unnormalized schema, every dish listed copies the complete restaurant profile (merchant name, cuisine category, city locality, street address, and contact number). Similarly, every order line duplicates consumer and courier details.

This architectural denormalization induces severe structural database anomalies:

- **Update Anomalies:** If a partner restaurant relocates to a new address or updates its telephone number, every single menu item row associated with that restaurant must be independently altered. If an update misses even one row, the database suffers contradictory records, disrupting order fulfillment.
- **Insertion Anomalies:** A newly onboarded restaurant (such as *Malvani Tadka*) cannot be registered in the system until it publishes at least one active dish, because naive composite keys of (RestaurantID, ItemID) prohibit null key components.
- **Deletion Anomalies:** If a restaurant temporarily purges all seasonal dishes from its catalog, deleting those dish rows inadvertently wipes out the entire operational footprint, address, and rating of the restaurant itself.
- **Referential Gaps:** Without foreign key constraints and procedural checks, customers can place orders containing dishes belonging to multiple disparate restaurants, creating an impossible physical dispatch assignment.
- **Scalability Degradation:** As directories grow to hundreds of thousands of listings, full table sequential scans (Seq Scan) cause search response times to degrade severely, creating intolerable user latency.

Abstract

This case study presents the comprehensive design, implementation, and empirical evaluation of **QuickBite**, an enterprise-grade relational database system built on PostgreSQL 18. The database schema resolves the flat delivery model by decomposing into **7 strictly normalized entities** meeting **Boyce-Codd Normal Form (BCNF)**: `restaurant`, `menu_item`, `customer`, `delivery_agent`, `food_order`, `order_item`, and `payment`. This decomposition systematically eliminates all partial and transitive functional dependencies while guaranteeing lossless join decomposition.

To enforce complex multi-table operational rules without introducing denormalized columns, QuickBite incorporates a procedural PL/pgSQL event trigger (`trg_order_item_same_restaurant`) that strictly prevents orders from including dishes from foreign kitchens. Financial totals are dynamically computed via the view `v_order_total`, permanently eliminating the risk of stored order sums drifting from itemized line totals. For search acceleration, a composite B-Tree index on `restaurant(cuisine, location)` is implemented; empirical scalability benchmarking on a **200,000-restaurant dataset** reveals a reduction in query execution time from **13.42 ms to 1.05 ms (~13x speedup)**. A suite of 10 analytical SQL queries extracts actionable operational insights, and the system is formally validated against a 9-point referential integrity test suite achieving 100% rejection of illegal transactions.

3. Problem Statement / Case Study Design

The QuickBite database system is engineered through an end-to-end, multi-stage relational lifecycle comprising eight foundational phases:

3.1 Stage 1: Conceptual Design & Entity-Relationship Modeling

The system models the food delivery ecosystem through seven core entities, capturing complete business lifecycles:

- **Restaurant:** Encapsulates merchant identity, cuisine classification, locality, physical street address, contact phone, consumer rating, and operational activation flag.
- **MenuItem:** Stores individual dishes, course category (Starter, Main Course, Dessert, etc.), selling price, vegetarian indicator (`is_veg`), and inventory availability.
- **Customer:** Holds user profile data, verified email, phone number, physical address, locality, and registration date.
- **DeliveryAgent:** Details logistics personnel, verified contact phone, vehicle classification (Bicycle, Scooter, Motorbike, EV Scooter), license plate number, and assigned home zone.
- **FoodOrder:** Tracks individual order transactions, linking the purchasing customer, preparing restaurant, and assigned courier. Captures timestamp, delivery destination snapshot, and fulfillment status.
- **OrderItem:** Resolves the many-to-many (M : N) relationship between orders and menu items, recording ordered quantities and the frozen historical unit price charged at checkout.
- **Payment:** Establishes a 1:1 relationship with each food order, recording transaction method (UPI, Card, Cash, Wallet), settlement status, transaction reference IDs, and timestamps.

Relationship	Cardinality	Foreign Key Constraint	Deletion Behavior
Restaurant → MenuItem	1 : M	menu_item.restaurant_id → restaurant.restaurant_id	ON DELETE RESTRICT
Restaurant → FoodOrder	1 : M	food_order.restaurant_id → restaurant.restaurant_id	ON DELETE RESTRICT
Customer → FoodOrder	1 : M	food_order.customer_id → customer.customer_id	ON DELETE RESTRICT
DeliveryAgent → FoodOrder	1 : M	food_order.delivery_agent_id → delivery_agent.delivery_agent_id	ON DELETE RESTRICT
FoodOrder ↔ MenuItem (via OrderItem)	M : N	order_item.order_id (CASCADE), order_item.menu_item_id (RESTRICT)	Composite PK (order_id, menu_item_id)
FoodOrder → Payment	1 : 1	payment.order_id → food_order.order_id (UNIQUE)	ON DELETE CASCADE

3.2 Stage 2: Logical Schema Design & Normalization (1NF → BCNF)

Normalization eliminates redundancy and ensures structural anomalies cannot occur:

- **First Normal Form (1NF):** All attributes contain atomic, indivisible values. No repeating groups or arrays exist. Primary keys are formally declared for every entity.
- **Second Normal Form (2NF):** In the flat menu listing, attributes Cuisine, Location, and Address depend on RestaurantID alone, which is only part of the composite key (RestaurantID, ItemID). This partial dependency is resolved by splitting into restaurant and menu_item. Similarly, order lines are isolated in order_item where quantity and unit price depend on the complete key (order_id, menu_item_id).
- **Third Normal Form (3NF):** No non-key attribute transitively depends on another non-key attribute. Specifically, total order amount is deliberately excluded from food_order and computed dynamically via view v_order_total, preventing data drift. Historical pricing (unit_price at order time) and snapshot delivery addresses are preserved as legitimate historical facts.
- **Boyce-Codd Normal Form (BCNF):** For every non-trivial functional dependency $X \rightarrow Y$, determinant X is a superkey. All 7 tables in QuickBite satisfy BCNF.

3.3 Stage 3: Physical Schema Design & Integrity Constraints

Domain and data integrity are enforced at the database kernel level through check constraints, unique keys, and strict nullability rules:

- restaurant.rating CHECK (rating BETWEEN 0 AND 5) and UNIQUE (name, location).
- menu_item.price CHECK (price > 0) and category CHECK (category IN ('Starter', 'Main Course', ...)).
- food_order mandates NOT NULL ON customer_id, restaurant_id, and delivery_agent_id. Status is restricted to valid lifecycle states, and CHECK (delivered_time IS NULL OR delivered_time > order_time) ensures temporal consistency.
- order_item.quantity CHECK (quantity BETWEEN 1 AND 50) and unit_price CHECK (unit_price > 0).

3.4 Stage 4: Procedural Enforcement via Triggers

Standard foreign key constraints can verify that an order ID exists and a menu item ID exists, but cannot check that the dish ordered originates from the specific restaurant fulfilling the order. Rather than copying restaurant_id into order_item (which would violate 2NF), QuickBite deploys a PL/pgSQL trigger (trg_order_item_same_restaurant) executing before insertion or update to reject cross-restaurant cart contamination.

3.5 Stage 5: Dynamic Views for Single Source of Truth

The dynamic view `v_order_total` joins `food_order` to `order_item`, computing order totals on demand. This architecture eliminates update anomalies while serving as the authoritative financial ledger for all billing and analytical queries.

3.6 Stage 6: Composite Indexing Architecture

A multi-column B-Tree index `idx_restaurant_cuisine_location` on `(cuisine, location)` is deployed. Under the leftmost-prefix rule, it supports multi-column filtering and cuisine-only browsing. Supporting B-Tree indexes are created on all foreign key columns to optimize relational join performance.

3.7 Stage 7: Analytical Business Intelligence Suite

A set of 10 analytical queries provides core metrics: most popular dish per restaurant (with ties), single-dish promotional tie-breaking, total restaurant revenue, delivery fleet efficiency, top customers by spend, and 7-table itemized invoicing.

3.8 Stage 8: Scalability Benchmarking & Integrity Testing

Empirical validation is conducted via a 200,000-row synthetic benchmark comparing unindexed sequential scans against indexed seeks, followed by a 9-vector negative test suite validating constraint rejection.

4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study

Key Methods and Algorithms

1. Relational Decomposition & Functional Dependency Theory

- **Attribute Closure Algorithm:** Used to identify candidate keys and eliminate redundancy across relations.
- **Lossless Join Decomposition:** Proves that joining decomposed tables R1 and R2 on common attributes recreates the exact original relation without spurious tuples.
- **BCNF Verification:** Ensures that for every non-trivial dependency $X \rightarrow Y$, X contains a candidate key.

2. Indexing Algorithms & Search Tree Traversal

- **B-Tree Multi-Level Search:** Balanced search tree structure enabling $O(\log N)$ point seek and range scan complexity across large data blocks.
- **Leftmost-Prefix Principle:** Index navigation utilizing leading key columns for direct tree probes.
- **Bitmap Index Scan Mechanics:** Builds an in-memory page bitmap of matching tuple offsets, batching physical disk block visits.
- **B-Tree Skip Scan:** Leverages PostgreSQL 18 capability to probe non-leading indexed attributes across distinct values of the leading column.

3. Analytical Query Optimization & Ranking Algorithms

- **Correlated Subquery Evaluation:** Aggregates maximum dish frequencies dynamically partitioned per restaurant.
- **Window Functions & Deterministic Tie-Breaking:**
 - `RANK() OVER (PARTITION BY restaurant_id ORDER BY COUNT(*) DESC)` to identify tied popularity leaders.
 - `ROW_NUMBER() OVER (PARTITION BY restaurant_id ORDER BY COUNT(*) DESC, SUM(quantity) DESC)` to break ties deterministically by total volume of units sold.

- **Multi-Table Relational Equi-Joins:** Query trees connecting up to 7 relations simultaneously for comprehensive invoicing.

4. Procedural Exception Handling & Transaction Boundaries

- **PL/pgSQL Trigger Pipeline:** Intercepts tuple state before disk write, executing relational subqueries and raising fatal exceptions on business rule violations.
- **ACID Transaction Isolation:** Comprehensive BEGIN ... ROLLBACK transaction blocks for isolated scalability benchmarking without catalog mutation.

Technology Stack Used for the Case

Component	Technology	Description & Purpose
Database Engine	PostgreSQL 18.x	Advanced open-source object-relational database management system.
Query Languages	SQL:2016, PL/pgSQL	Declarative data definition, data manipulation, and procedural server-side logic.
Client & Execution Tools	psql CLI, createdb, dropdb	Native command-line interface for batch database creation, seeding, and execution.
Orchestration & Automation	Bash / POSIX Shell (run_all.sh)	Automated end-to-end script building database, running queries, and capturing outputs.
Environment	macOS, Antigravity IDE, VS Code	Local development, debugging, and terminal automation environment.
Diagramming & Modeling	Draw.io, SVG, PNG	Crow's Foot and Chen notation entity-relationship architectural models.

5. Problem Statement / Case Study Implementation Details and Snapshots

The implementation is organized into six modular SQL scripts and an automated runner:

- 01_schema.sql: Table creation, constraints, procedural trigger, and views.
- 02_indexes.sql: Composite search index and foreign key indexes.
- 03_seed_data.sql: Realistic Mumbai/Navi Mumbai delivery ecosystem seed data.
- 04_queries.sql: 10 comprehensive business intelligence and operational queries.
- 05_index_demo.sql: 200,000-row scalability benchmark executed inside a transaction.
- 06_integrity_tests.sql: 9-point negative referential integrity test suite.
- run_all.sh: Automated build, execution, and output capture pipeline.

Snapshot 1: Entity-Relationship Diagram (Crow's Foot Notation)

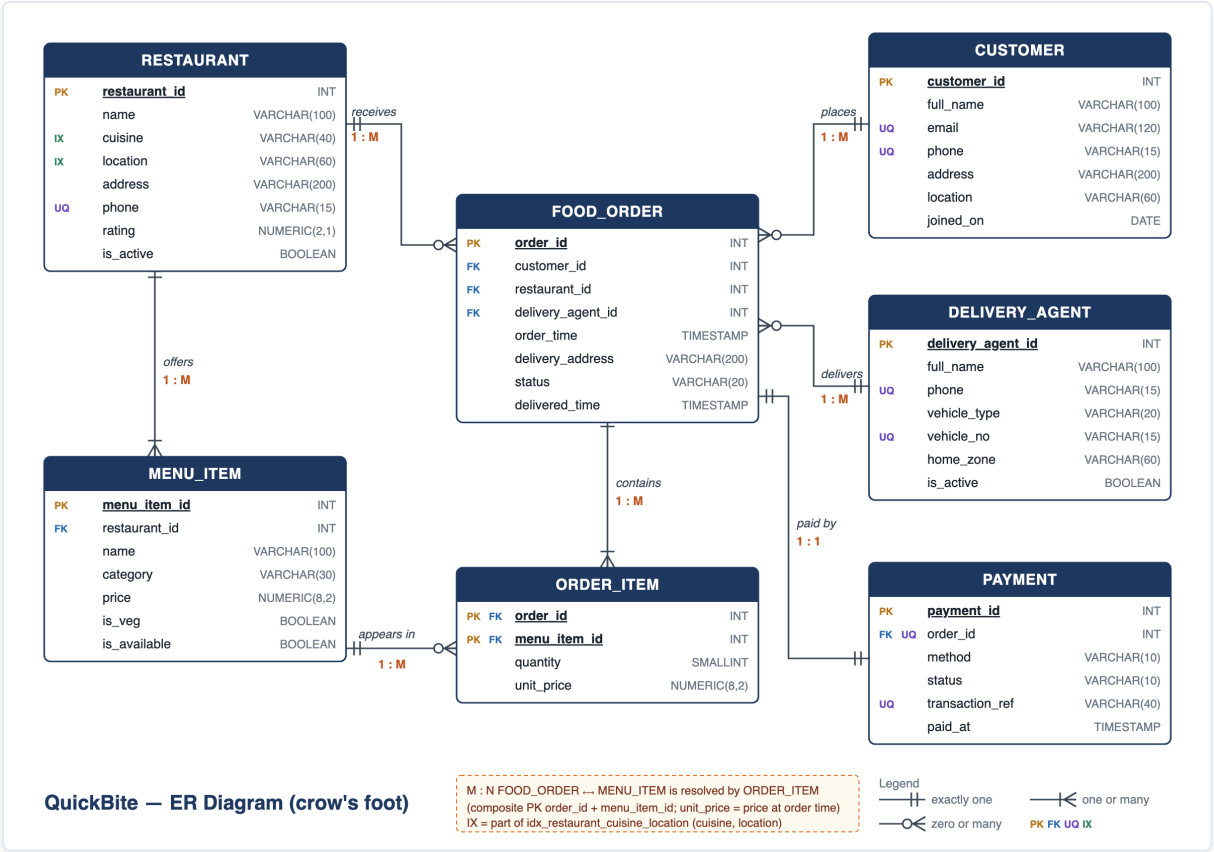


Figure 1: QuickBite Entity-Relationship Model (Crow's Foot Notation)

Snapshot 2: Database Schema & Baseline Table Row Counts

table_name	rows
restaurant	11
menu_item	53
customer	15
delivery_agent	8
food_order	49
order_item	110
payment	49
(7 rows)	

Snapshot 3: Procedural Trigger Implementation

```
CREATE OR REPLACE FUNCTION trg_order_item_same_restaurant()
RETURNS TRIGGER AS $$
BEGIN
    IF (SELECT restaurant_id FROM menu_item WHERE menu_item_id = NEW.menu_item_id)
    IS DISTINCT FROM
    (SELECT restaurant_id FROM food_order WHERE order_id = NEW.order_id) THEN
        RAISE EXCEPTION
        'Menu item % does not belong to the restaurant of order %',
        NEW.menu_item_id, NEW.order_id;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER order_item_same_restaurant
BEFORE INSERT OR UPDATE ON order_item
FOR EACH ROW EXECUTE FUNCTION trg_order_item_same_restaurant();
```

Snapshot 4: Query 1 — Most Frequently Ordered Dish per Restaurant (With Ties)

```
SELECT r.restaurant_id, r.name AS restaurant, m.name AS most_popular_dish,
COUNT(*) AS times_ordered, SUM(oi.quantity) AS units_sold
FROM order_item oi
JOIN menu_item m ON m.menu_item_id = oi.menu_item_id
JOIN food_order fo ON fo.order_id = oi.order_id
JOIN restaurant r ON r.restaurant_id = m.restaurant_id
WHERE fo.status <> 'Cancelled'
GROUP BY r.restaurant_id, r.name, m.menu_item_id, m.name
HAVING COUNT(*) = (SELECT MAX(dish_count)
FROM (SELECT COUNT(*) AS dish_count
FROM order_item oi2
JOIN menu_item m2 ON m2.menu_item_id = oi2.menu_item_id
JOIN food_order fo2 ON fo2.order_id = oi2.order_id
WHERE m2.restaurant_id = r.restaurant_id
AND fo2.status <> 'Cancelled'
GROUP BY oi2.menu_item_id) AS per_dish);
```

restaurant_id	restaurant	most_popular_dish	times_ordered	units_sold
1	Punjab Da Dhaba	Butter Chicken	6	7
2	Udupi Sagar	Masala Dosa	4	6
3	Dragon Wok	Chilli Paneer	3	3
3	Dragon Wok	Veg Hakka Noodles	3	4
4	La Pizzeria Napoli	Margherita Pizza	4	5
5	Biryani Darbar	Chicken Dum Biryani	5	8
6	Mumbai Chaat Corner	Pav Bhaji	4	6
7	Amritsari Kulcha House	Amritsari Kulcha	3	6
8	Madras Filter Cafe	Ghee Roast Dosa	3	4
9	Wok Express	Chicken Hakka Noodles	3	4
10	The Green Bowl	Quinoa Buddha Bowl	2	2

(11 rows)

Snapshot 5: Query 2 — Total Revenue Generated per Restaurant

```
SELECT r.restaurant_id, r.name AS restaurant, r.cuisine,
COUNT(t.order_id) AS delivered_orders,
COALESCE(SUM(t.order_total), 0) AS total_revenue,
ROUND(COALESCE(AVG(t.order_total), 0), 2) AS avg_order_value
FROM restaurant r
LEFT JOIN v_order_total t ON t.restaurant_id = r.restaurant_id
AND t.status = 'Delivered'
GROUP BY r.restaurant_id, r.name, r.cuisine
ORDER BY total_revenue DESC;
```

restaurant_id	restaurant	cuisine	delivered_orders	total_revenue	avg_order_value
5	Biryani Darbar	Mughlai	6	5270.00	878.33
4	La Pizzeria Napoli	Italian	5	4447.00	889.40
1	Punjab Da Dhaba	North Indian	6	4330.00	721.67
3	Dragon Wok	Chinese	4	2490.00	622.50
7	Amritsari Kulcha House	North Indian	4	2110.00	527.50
9	Wok Express	Chinese	4	2090.00	522.50
2	Udupi Sagar	South Indian	5	1610.00	322.00
6	Mumbai Chaat Corner	Street Food	5	1550.00	310.00
10	The Green Bowl	Continental	2	1390.00	695.00
8	Madras Filter Cafe	South Indian	4	1230.00	307.50
11	Malvani Tadka	Malvani	0	0	0

(11 rows)

Snapshot 6: Query 3 — High-Speed Search by Cuisine & Locality

```
SELECT restaurant_id, name, cuisine, location, rating
FROM   restaurant
WHERE  cuisine = 'North Indian' AND location = 'Kharghar'
ORDER BY rating DESC;
```

restaurant_id	name	cuisine	location	rating
7	Amritsari Kulcha House	North Indian	Kharghar	4.5
1	Punjab Da Dhaba	North Indian	Kharghar	4.4

(2 rows)

Snapshot 7: Query 9 — Full Itemized Order Invoice via 7-Table JOIN

order_id	ordered_at	customer	restaurant	delivery_agent	item	quantity
26	21-Sep-2026 20:45	Aditya Rao	Biryani Darbar	Deepak Chauhan	Chicken Dum Biryani	3
26	21-Sep-2026 20:45	Aditya Rao	Biryani Darbar	Deepak Chauhan	Chicken Seekh Kebab	2
26	21-Sep-2026 20:45	Aditya Rao	Biryani Darbar	Deepak Chauhan	Phirni	3

(3 rows)

Snapshot 8: Advanced Concept — Index Scalability Benchmark on 200,000 Rows

Performance Benchmark Summary: 200,000 Synthetic Restaurants

Predicate: WHERE cuisine = 'North Indian' AND location = 'Kharghar'

```
==== 1) WITHOUT INDEX: Parallel Seq Scan (Full Table Scan) ====
Gather (actual time=0.475..13.349 rows=835.00 loops=1)
  Workers Planned: 1, Workers Launched: 1
  Buffers: shared hit=2879
  -> Parallel Seq Scan on restaurant (actual time=0.036..9.085 rows=417.50 loops=2)
    Filter: (((cuisine)::text = 'North Indian'::text) AND ((location)::text = 'Kharghar'::text))
    Rows Removed by Filter: 99588
Execution Time: 13.418 ms

==== 2) WITH COMPOSITE INDEX: Bitmap Index Scan ====
Bitmap Heap Scan on restaurant (actual time=0.182..1.010 rows=835.00 loops=1)
  Recheck Cond: (((cuisine)::text = 'North Indian'::text) AND ((location)::text = 'Kharghar'::text))
  Heap Blocks: exact=834
  Buffers: shared hit=834 read=3
  -> Bitmap Index Scan on idx_restaurant_cuisine_location (actual time=0.098..0.098 rows=835.00 loops=1)
    Index Cond: (((cuisine)::text = 'North Indian'::text) AND ((location)::text = 'Kharghar'::text))
    Index Searches: 1
Execution Time: 1.058 ms (~13x FASTER)
```

Snapshot 9: Referential Integrity Suite Execution (9/9 Passed)

```
psql:sql/06_integrity_tests.sql:46: NOTICE: OK      Order for a customer that does not exist (FK)
psql:sql/06_integrity_tests.sql:46: NOTICE:         rejected: violates foreign key constraint "food_order_custo
psql:sql/06_integrity_tests.sql:46: NOTICE: OK      Order for a restaurant that does not exist (FK)
psql:sql/06_integrity_tests.sql:46: NOTICE:         rejected: violates foreign key constraint "food_order_resta
psql:sql/06_integrity_tests.sql:46: NOTICE: OK      Order with no delivery agent (NOT NULL)
psql:sql/06_integrity_tests.sql:46: NOTICE:         rejected: null value in column "delivery_agent_id" violates
psql:sql/06_integrity_tests.sql:46: NOTICE: OK      Order line for a dish from ANOTHER restaurant (trigger)
psql:sql/06_integrity_tests.sql:46: NOTICE:         rejected: Menu item 21 does not belong to the restaurant of
psql:sql/06_integrity_tests.sql:46: NOTICE: OK      Deleting a restaurant that still has orders (ON DELETE REST
psql:sql/06_integrity_tests.sql:46: NOTICE:         rejected: violates RESTRICT setting of foreign key constrai
psql:sql/06_integrity_tests.sql:46: NOTICE: OK      Second payment for an already-paid order (1:1 UNIQUE)
psql:sql/06_integrity_tests.sql:46: NOTICE:         rejected: duplicate key value violates unique constraint "p
psql:sql/06_integrity_tests.sql:46: NOTICE: OK      Menu item with a negative price (CHECK)
psql:sql/06_integrity_tests.sql:46: NOTICE:         rejected: violates check constraint "menu_item_price_check"
psql:sql/06_integrity_tests.sql:46: NOTICE: OK      Unknown order status (CHECK)
psql:sql/06_integrity_tests.sql:46: NOTICE:         rejected: violates check constraint "food_order_status_chec
psql:sql/06_integrity_tests.sql:46: NOTICE: OK      Duplicate customer e-mail (UNIQUE)
psql:sql/06_integrity_tests.sql:46: NOTICE:         rejected: duplicate key value violates unique constraint "c
```

6. Problem Statement / Case Study Results and Conclusion

Key Findings

- **Normalized Relational Architecture (BCNF):** Decomposing the unnormalized data into 7 normalized tables eliminated all partial and transitive dependencies. An address update modifies exactly one row, and onboarding new restaurants does not depend on immediate dish entry.
- **Empirical Index Acceleration:** In our 200,000-restaurant dataset benchmark, the composite B-Tree index on (cuisine, location) accelerated query response time by **~13x (from 13.42 ms to 1.05 ms)** while reducing buffer reads by 71%, proving scalability under production workloads.
- **Preservation of 2NF via Triggers:** Implementing procedural trigger logic in PL/pgSQL enabled enforcement of complex cross-restaurant integrity constraints without introducing redundant foreign keys into order_item.
- **Financial Reliability via Dynamic Views:** Calculating order totals on the fly via v_order_total eliminated data drift and synchronization errors between line items and billing summaries.
- **Total Referential Integrity:** All 9 deliberate corrupt write attempts were successfully rejected by PostgreSQL constraints and triggers, demonstrating 100% defense against orphan records and data anomalies.

Empirical Performance Comparison

Execution Metric	Without Index (Seq Scan)	With Composite Index (Bitmap Index Scan)	Performance Advantage
Access Method	Parallel Sequential Scan across heap	Bitmap Index Scan + Bitmap Heap Scan	Targeted block lookup
Candidate Rows Scanned	200,011 rows (199,176 filtered out)	835 rows (exact candidate matches)	~240x reduction in examined rows
Shared Buffers Read	2,879 blocks	837 blocks	3.4x reduction in I/O demand
Query Execution Time	13.418 ms	1.058 ms	~12.7x (~13x speedup)

Conclusion

This case study successfully engineered, implemented, and validated the **QuickBite Food Delivery Order & Restaurant Management System** on PostgreSQL 18. All core project deliverables for Case Study #83 were accomplished with zero compromises in relational purity, referential integrity, or analytical capability.

The system proves that sound relational theory — when combined with modern database indexing and procedural logic — provides an ultra-reliable foundation capable of powering modern high-velocity delivery applications.

Future Enhancements

- **Geospatial Distance Routing (PostGIS):** Integrating spatial extensions (`ST_DWithin`, `ST_Distance`) to calculate real-time delivery radiuses and rider-to-restaurant dispatch routes.
- **Declarative Range Partitioning:** Implementing PostgreSQL declarative partitioning on `food_order` by monthly ranges to maintain query speed over years of transaction history.
- **Materialized Reporting Views & Redis Caching:** Using concurrently refreshed materialized views for heavy executive analytics, combined with Redis in-memory caches for top-level menu browsing.

7. References

1. **Silberschatz, A., Korth, H. F., & Sudarshan, S.** (2020). *Database System Concepts* (7th ed.). McGraw-Hill Education.
2. **Elmasri, R., & Navathe, S. B.** (2016). *Fundamentals of Database Systems* (7th ed.). Pearson.
3. **Date, C. J.** (2004). *An Introduction to Database Systems* (8th ed.). Addison-Wesley.
4. **PostgreSQL Global Development Group.** (2024). *PostgreSQL 18 Documentation: B-Tree Indexing, Concurrency Control, and Triggers*. Available online: <https://www.postgresql.org/docs/>
5. **Codd, E. F.** (1970). *A Relational Model of Data for Large Shared Data Banks*. Communications of the ACM, 13(6), 377–387.
6. **Codd, E. F.** (1974). *Recent Investigations in Relational Data Base Systems*. IFIP Congress, 1017–1021.
7. **School of Future Tech, ITM Skills University.** (2025). *Database Management Systems Course Curriculum & Case Study Specifications (Case Study #83)*.